

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Базы данных

ОТЧЕТ
по лабораторной работе № 3
на тему
«ЖД-вокзал»
Разработка NoSQL базы данных и спецификаций прикладной программы

Студент:

Д. А. Снитко

Преподаватель:

С. С. Силич

МИНСК 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ	4
2 ПРОГРАММИРОВАНИЕ КОНВЕРТЕРА ДАННЫХ.....	6
2.1 Описание функционала	6
2.2 Описание ключевых методов.....	6
2.3 Тестирование конвертера.....	7
2.3.1 Запуск конвертации	8
2.3.2 Просмотр списка таблиц	8
2.3.3 Просмотр содержимого таблицы	9
2.3.4 Сохранение таблицы в файл	9
2.3.5 Содержимое файла с сохраненной таблицей	10
ЗАКЛЮЧЕНИЕ	11
ПРИЛОЖЕНИЕ А	12

ВВЕДЕНИЕ

Данная работа посвящена разработке программного комплекса для конвертации данных из реляционной базы данных PostgreSQL в нереляционную key-value базу данных Berkeley DB. Целью является создание утилиты, способной автоматически переносить структуру и данные, а также интерактивного меню для управления процессом и просмотра результатов.

Предметная область включает динамическое чтение схемы данных из PostgreSQL (таблицы, первичные ключи, столбцы) и их последующую сериализацию в формат JSON. Ключи в Berkeley DB формируются на основе первичных ключей исходных таблиц, обеспечивая быстрый доступ к данным.

Программный комплекс состоит из трех основных модулей. Конвертер. Ядро системы, отвечающее за подключение к PostgreSQL, анализ схемы и перенос данных.

Интерактивное меню. Консольное приложение, предоставляющее пользователю интерфейс для запуска конвертации и просмотра данных в Berkeley DB.

Просмотрщик. Утилита для вывода содержимого баз данных Berkeley DB, используемая меню для отображения и сохранения данных.

Обмен данными внутри Berkeley DB выполняется в формате UTF-8, где значения представляют собой JSON-строки, обеспечивая гибкость и совместимость. Система также предусматривает возможность выгрузки данных из Berkeley DB в excel для дальнейшего анализа или резервного копирования.

1 ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ

Данные из PostgreSQL сохраняются в Berkeley DB по следующей обобщенной схеме:

- Каждой таблице из схемы `public` в PostgreSQL соответствует отдельная база данных в окружении Berkeley DB.
- Ключом для записи в Berkeley DB является строковое представление значения первичного ключа.
- Значением является JSON-объект, содержащий все столбцы строки, за исключением тех, что вошли в первичный ключ.

Конвертер `PgToBerkeley.java` работает по следующему алгоритму:

1. Читает конфигурационные данные для подключения к PostgreSQL.
2. Устанавливает соединение с базой данных PostgreSQL с помощью JDBC.
3. Получает список всех таблиц в схеме `public`.
4. Для каждой таблицы определяет её первичные ключи и остальные столбцы.
5. Создает или открывает окружение Berkeley DB в указанном каталоге.
6. Для каждой таблицы создает соответствующую базу данных в окружении Berkeley DB.
7. Выполняет SQL-запрос `SELECT * FROM table` для получения всех строк.
8. Для каждой строки формирует ключ на основе значений первичных ключей и JSON-значение из остальных столбцов.
9. Записывает пару "ключ-значение" в соответствующую базу данных Berkeley DB.
10. После обработки всех таблиц закрывает все соединения.

В таблице 1.1 представлено описание обобщенной структуры хранения данных в Berkeley DB.

Таблица 1.1 – Структура хранения данных модели «ЖД-вокзал»

Имя таблицы	Ключ	Значение
Билет	id	Место, Время_покупки, Стоимость, Категория
Маршрут	id	Время_в_пути, Количество_остановок, Название, Расстояние
Маршрут_Станция	id_Маршрут_id_Станция_id_Сотрудник_id	Порядок_остановки
Пассажир	id	ФИО, Номер_паспорта, Пол, Контактный_телефон

Продолжение таблицы 1.1

Поезд	id	Количество_вагонов, Вместимость, Компания, Тип
Поезд_Сотрудник	id, Поезд_id_Сотрудник_id	—
Расписание	id, Поезд_id_Маршрут_id_Станция_отбытия_id_Станция_прибытия_id	Время_отбытия, Время_прибытия
Сотрудник	id	ФИО, Контактный_телефон, Должность, Стаж
Сотрудник_Маршрут	id_Сотрудник_id_Маршрут_id	—
Станция	id	Название, Город, Количество_платформ, Количество_путей

В таблице 1.2 представлен пример хранения данных в Berkeley DB для таблицы «Пассажир».

Таблица 1.2 – Пример хранения данных в Berkeley DB для таблицы «Пассажир»

Имя таблицы	Ключ	Значение (JSON)
Пассажир	1	{ "ФИО": "Иванов Иван Иванович", "Номер_паспорта": "1234 567890", "Пол": "Мужской", "Контактный_телефон": "+79001234567" }
Пассажир	2	{ "ФИО": "Петрова Анна Сергеевна", "Номер_паспорта": "5678 123456", "Пол": "Женский", "Контактный_телефон": "+79017654321" }

2 ПРОГРАММИРОВАНИЕ КОНВЕРТЕРА ДАННЫХ

2.1 Описание функционала

Данный конвертер реализует конвертер данных, который переводит данные из базы данных PostgreSQL в Berkeley DB.

Проект состоит из нескольких ключевых файлов, обеспечивающих его функционирование.

`PgToBerkeley.java`. Основной модуль, реализующий логику конвертации данных. Он подключается к PostgreSQL, запрашивает схему данных, читает строки из таблиц и записывает их в Berkeley DB в формате "ключ-значение". Ключ формируется из первичного ключа таблицы, а значение — из остальных полей в формате JSON.

`AppMenu.java`. Консольное меню, служащее точкой входа для пользователя. Оно предоставляет следующие опции:

- Запуск конвертера `PgToBerkeley.java`;
- Просмотр списка созданных таблиц в Berkeley DB;
- Отображение содержимого конкретной таблицы;
- Сохранение данных таблицы в текстовый файл.

Для выполнения операций просмотра и сохранения меню использует утилиту `DumpBerkeley.java`.

`DumpBerkeley.java`. Вспомогательная утилита для работы с данными в Berkeley DB. Она может выводить список всех баз данных (таблиц) в окружении или печатать содержимое указанной базы данных в консоль или файл.

`config.json`. Файл конфигурации, который используется `AppMenu.java` для определения пути к окружению Berkeley DB (`berkeley_data`).

2.2 Описание ключевых методов

Файл `PgToBerkeley.java`.

– `main(String[] args)`: точка входа в конвертер. Устанавливает соединение с PostgreSQL, используя параметры из конфигурации, и инициирует процесс конвертации всех таблиц.

– `convertAllTables(Connection conn, Path envDir)`: оркестрирует процесс конвертации. Получает список таблиц, и для каждой из них создает отдельную базу данных в окружении Berkeley DB, после чего вызывает методы для построения ключей и значений и записывает их.

– `listPublicTables(Connection conn)`: запрашивает у системного каталога PostgreSQL (`information_schema.tables`) список всех таблиц, принадлежащих схеме `public`.

- `listPrimaryKeyColumns(Connection conn, String table)`: определяет, какие столбцы составляют первичный ключ для указанной таблицы.
- `buildKey(...)`: формирует строковый ключ для записи в Berkeley DB. Если первичный ключ состоит из нескольких столбцов, их значения конкатенируются через `_`.

- `buildJson(...)`: создает JSON-строку, содержащую все поля строки из `ResultSet`, которые не являются частью первичного ключа.

Файл `AppMenu.java`.

- `main(String[] args)`: главный метод, запускающий интерактивное меню. Организует цикл, в котором отображаются опции, считывается ввод пользователя и вызываются соответствующие функции.

- `runConverter(Path baseDir)`: запускает конвертер `PgToBerkeley.java` в отдельном процессе. Это предотвращает завершение работы меню, если конвертер вызовет `System.exit()`.

- `listTables(String env)`: вызывает утилиту `DumpBerkeley.java` с флагом `--list` для отображения списка всех таблиц, хранящихся в окружении Berkeley DB.

- `showTable(String env, String table)`: вызывает `DumpBerkeley.java`, передавая ей имя таблицы, для вывода ее содержимого в консоль.

- `saveTable(String env, String table, Path outFile)`: вызывает `DumpBerkeley.java` для сохранения данных таблицы, перенаправляя ее стандартный вывод в указанный файл.

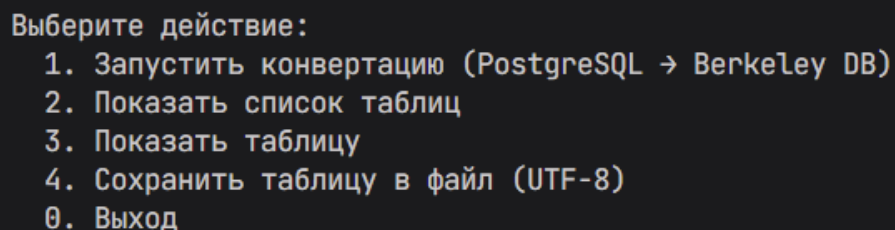
Файл `DumpBerkeley.java`.

- `main(String[] args)`: основной метод утилиты просмотра. Анализирует аргументы командной строки. Если указан флаг `--list`, выводит имена всех баз данных в окружении. Если указано имя базы, выводит ее содержимое (пары ключ-значение) в консоль или в файл.

2.3 Тестирование конвертера

Для проверки работоспособности конвертера используется интерактивное меню `AppMenu.java`, которое позволяет шаг за шагом выполнить все ключевые операции.

После запуска программы пользователю отображается меню представленное на рисунке 2.1



```
Выберите действие:
1. Запустить конвертацию (PostgreSQL → Berkeley DB)
2. Показать список таблиц
3. Показать таблицу
4. Сохранить таблицу в файл (UTF-8)
0. Выход
```

Рисунок 2.1 – Меню программы

2.3.1 Запуск конвертации

При выборе пункта меню 1 запускает процесс конвертации, выполняемый классом PgToBerkeley. Процесс подключается к PostgreSQL и переносит данные в Berkeley DB.

```
Ваш выбор: 1
→ Запуск конвертации...
[PgToBerkeley] Conversion start...
[PgToBerkeley] Connecting to jdbc:postgresql://localhost:5432/Railway
[PgToBerkeley] Table Билет -> rows: 31
[PgToBerkeley] Table Маршрут -> rows: 31
[PgToBerkeley] Table Маршрут_Станция -> rows: 31
[PgToBerkeley] Table Пассажир -> rows: 31
[PgToBerkeley] Table Поезд -> rows: 31
[PgToBerkeley] Table Поезд_Сотрудник -> rows: 31
[PgToBerkeley] Table Расписание -> rows: 31
[PgToBerkeley] Table Сотрудник -> rows: 31
[PgToBerkeley] Table Сотрудник_Маршрут -> rows: 31
[PgToBerkeley] Table Станция -> rows: 31
[PgToBerkeley] Conversion finished successfully.
✓ Конвертация завершена
Нажмите Enter, чтобы вернуться в меню...
```

Рисунок 2.2 – Результат выполнения конвертации

2.3.2 Просмотр списка таблиц

Этот пункт меню вызывает DumpBerkeley для сканирования окружения Berkeley DB и выводит список всех доступных баз данных, которые соответствуют таблицам из PostgreSQL.

```
Ваш выбор: 2
→ Список таблиц (env=C:\Users\dw800\YandexDisk\7TERM\БД\Labs\Lab3\.\berkeley_data):
Билет
Маршрут
Маршрут_Станция
Пассажир
Поезд
Поезд_Сотрудник
Расписание
Сотрудник
Сотрудник_Маршрут
Станция
Нажмите Enter, чтобы вернуться в меню...
```

Рисунок 2.3 – Список таблиц, доступных в Berkeley DB

2.3.3 Просмотр содержимого таблицы

Программа запрашивает у пользователя имя таблицы, после чего выводит ее содержимое. Каждая запись представлена в виде ключа и JSON-значения.

```
Ваш выбор: 3
Введите имя таблицы: Маршрут
→ Печать таблицы 'Маршрут' (env=C:\Users\dw800\YandexDisk\7TERM\БД\Labs\Lab3\.\berkeley_data
):
VERSION=viewer
db=Маршрут
HEADER=END
1
{"Время_в_пути":"0 years 0 mons 0 days 1 hours 20 mins 0.0 secs","Количество_остановок":2,"На
азвание":"Минск - Молодечно","Расстояние":75}
10
{"Время_в_пути":"0 years 0 mons 0 days 1 hours 30 mins 0.0 secs","Количество_остановок":2,"На
азвание":"Гродно - Лида","Расстояние":115}
11
{"Время_в_пути":"0 years 0 mons 0 days 2 hours 0 mins 0.0 secs","Количество_остановок":3,"На
азвание":"Брест - Пинск","Расстояние":180}
12
{"Время_в_пути":"0 years 0 mons 0 days 4 hours 50 mins 0.0 secs","Количество_остановок":6,"На
азвание":"Минск - Калининград","Расстояние":530}
13
{"Время_в_пути":"0 years 0 mons 0 days 5 hours 10 mins 0.0 secs","Количество_остановок":7,"На
азвание":"Минск - Смоленск","Расстояние":340}
14
{"Время_в_пути":"0 years 0 mons 0 days 2 hours 40 mins 0.0 secs","Количество_остановок":3,"На
азвание":"Могилёв - Орша","Расстояние":140}
```

Рисунок 2.4 – Просмотр содержимого конкретной таблицы (Маршрут)

2.3.4 Сохранение таблицы в файл

Данная опция позволяет сохранить содержимое выбранной таблицы в текстовый файл. Программа запрашивает имя таблицы и путь для сохранения.

```
Ваш выбор: 4
Имя таблицы: Маршрут
Путь файла (например, view_Маршрут.txt): Маршрут
→ Сохранение в файл: C:\Users\dw800\YandexDisk\7TERM\БД\Labs\Lab3\Маршрут
✓ Готово
Нажмите Enter, чтобы вернуться в меню...█
```

Рисунок 2.5 – Сообщение об успешном сохранении таблицы в файл

2.3.5 Содержимое файла с сохраненной таблицей

Можно открыть сохраненный файл с содержащейся в нем сохраненной таблицей.

```
Lab3 > Маршрут
1  VERSION=viewer
2  db=Маршрут
3  HEADER=END
4  1
5  {"Время_в_пути":"0 years 0 mons 0 days 1 hours 20 mins 0.0 secs","Количество_остановок":2,"Название":"Минс
6  10
7  {"Время_в_пути":"0 years 0 mons 0 days 1 hours 30 mins 0.0 secs","Количество_остановок":2,"Название":"Грод
8  11
9  {"Время_в_пути":"0 years 0 mons 0 days 2 hours 0 mins 0.0 secs","Количество_остановок":3,"Название":"Брест
10 12
11 {"Время_в_пути":"0 years 0 mons 0 days 4 hours 50 mins 0.0 secs","Количество_остановок":6,"Название":"Минс
12 13
13 {"Время_в_пути":"0 years 0 mons 0 days 5 hours 10 mins 0.0 secs","Количество_остановок":7,"Название":"Минс
14 14
15 {"Время_в_пути":"0 years 0 mons 0 days 2 hours 40 mins 0.0 secs","Количество_остановок":3,"Название":"Могил
16 15
17 {"Время_в_пути":"0 years 0 mons 0 days 3 hours 20 mins 0.0 secs","Количество_остановок":4,"Название":"Гоме
18 16
19 {"Время_в_пути":"0 years 0 mons 0 days 5 hours 40 mins 0.0 secs","Количество_остановок":7,"Название":"Вите
20 17
21 {"Время_в_пути":"0 years 0 mons 0 days 4 hours 30 mins 0.0 secs","Количество_остановок":5,"Название":"Минс
22 18
23 {"Время_в_пути":"0 years 0 mons 0 days 7 hours 0 mins 0.0 secs","Количество_остановок":9,"Название":"Минск
24 19
25 {"Время_в_пути":"0 years 0 mons 0 days 3 hours 50 mins 0.0 secs","Количество_остановок":5,"Название":"Минс
26 2
```

Рисунок 2.6 – Содержимое файла с сохраненной таблицей

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы был разработан программный комплекс для преобразования данных из реляционной модели PostgreSQL в формат "ключ-значение" базы данных Berkeley DB. Этот процесс включал динамический анализ схемы, сериализацию данных в JSON и создание интерактивного меню для управления процессом.

Разработанные утилиты позволяют автоматизировать перенос данных и предоставляют удобные средства для их последующего просмотра и выгрузки.

Выполненная работа позволила глубже понять различия между реляционными и нереляционными СУБД, а также освоить практические навыки работы с JDBC и встраиваемыми базами данных на языке Java.

Berkeley DB подходит для сценариев, где важна простота доступа к данным и минимальные требования к ресурсам, в то время как PostgreSQL остаётся предпочтительным выбором для приложений с более сложными требованиями к связям между данными и выполнению запросов.

ПРИЛОЖЕНИЕ А

(обязательное)

Код программы

Файл AppMenu.java

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;
import java.net.*;

public class AppMenu {
    private static final String DEFAULT_ENV = "berkeley_data";
    private static final String CONFIG_FILE = "config.json";

    private static Path getBaseDir() {
        try {
            Path loc =
Paths.get(AppMenu.class.getProtectionDomain().getCodeSource().getLocation().toURI());
            // Если класс загружен из каталога (например, Lab3),
используем его напрямую
            if (Files.isDirectory(loc)) {
                return loc;
            }
            // Иначе это путь к JAR/файлу — используем его родитель
как базу
            Path parent = loc.getParent();
            return parent != null ? parent :
Paths.get(System.getProperty("user.dir"));
        } catch (Exception e) {
            // Фолбек: если есть подкаталог Lab3 в текущей директории
— используем его
            Path cwd = Paths.get(System.getProperty("user.dir"));
            Path lab3 = cwd.resolve("Lab3");
            return Files.isDirectory(lab3) ? lab3 : cwd;
        }
    }

    private static String resolveEnv(Path baseDir) {
        Path cfg = baseDir.resolve(CONFIG_FILE);
        if (Files.exists(cfg)) {
            try {
                String text = Files.readString(cfg,
StandardCharsets.UTF_8);
                // Простой разбор без зависимостей
                java.util.regex.Matcher m = java.util.regex.Pattern
.compile("\"berkeley_env\"\\s*:\\s*\"([^\"]+)\"");
                .matcher(text);
                if (m.find()) {
                    String val = m.group(1).trim();
                    if (val.isEmpty()) {
```

```

        return
baseDir.resolve(DEFAULT_ENV).toString();
    }
    Path p = Paths.get(val);
    return p.isAbsolute() ? p.toString() :
baseDir.resolve(val).toString();
    }
    } catch (IOException ignored) {}
    }
    return baseDir.resolve(DEFAULT_ENV).toString();
}

private static void runConverter(Path baseDir) {
    // Запускаем конвертер во внешнем процессе, чтобы System.exit
    // внутри него не завершал меню
    try {
        String javaHome = System.getProperty("java.home");
        Path javaBin = Paths.get(javaHome, "bin", "java.exe");
        String javaExe = Files.exists(javaBin) ?
javaBin.toString() : "java";

        // Собираем classpath: Lab3 + все JAR'ы из Lab3/lib
        StringBuilder cp = new StringBuilder(baseDir.toString());
        Path lib = baseDir.resolve("lib");
        if (Files.isDirectory(lib)) {
            try (DirectoryStream<Path> stream =
Files.newDirectoryStream(lib, "*.jar")) {
                for (Path p : stream) {
                    cp.append(';').append(p.toAbsolutePath());
                }
            }
        }

        List<String> cmd = new ArrayList<>();
        cmd.add(javaExe);
        cmd.add("-Dfile.encoding=UTF-8");
        cmd.add("-cp");
        cmd.add(cp.toString());
        cmd.add("PgToBerkeley");

        ProcessBuilder pb = new ProcessBuilder(cmd);
        Path workDir = baseDir.getParent() != null ?
baseDir.getParent() : baseDir;
        pb.directory(workDir.toFile());
        // Включаем Unicode-логи конвертера
        Map<String, String> env = pb.environment();
        env.put("PG2B_LOGS", "utf8");
        pb.redirectErrorStream(true);
        Process proc = pb.start();
        try (BufferedReader br = new BufferedReader(new
InputStreamReader(proc.getInputStream(), StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        }
    }
}

```

```

        int code = proc.waitFor();
        if (code != 0) {
            System.out.println("Конвертер завершился с кодом: " +
code);
            System.out.println("Если видите ошибки драйвера (No
suitable driver), добавьте postgresql-*.jar в 'Lab3/lib'.");
        }
        } catch (Throwable e) {
            System.out.println("Ошибка запуска конвертера: " +
e.getMessage());
            System.out.println("Проверьте зависимости: требуется JAR
Berkeley JE (например, je-18.3.12.jar) и драйвер PostgreSQL (например,
postgresql-42.7.3.jar). Поместите их в папку 'Lab3/lib' и запустите
снова.");
        }
    }

    private static void listTables(String env) {
        if (!Files.exists(Paths.get(env))) {
            System.out.println("Среда Berkeley DB не найдена: " +
Paths.get(env).toAbsolutePath());
            System.out.println("Запустите сначала конвертацию (пункт
1), чтобы создать данные.");
            return;
        }
        try {
            DumpBerkeley.main(new String[]{"--env", env, "--list"});
        } catch (Throwable e) {
            System.out.println("Ошибка списка таблиц: " +
e.getMessage());
            System.out.println("Вероятно, не найден JAR Berkeley JE.
Поместите je-*.jar в папку 'Lab3/lib'.");
        }
    }

    private static void showTable(String env, String table) {
        if (!Files.exists(Paths.get(env))) {
            System.out.println("Среда Berkeley DB не найдена: " +
Paths.get(env).toAbsolutePath());
            System.out.println("Запустите сначала конвертацию (пункт
1).");
            return;
        }
        try {
            DumpBerkeley.main(new String[]{"--env", env, table});
        } catch (Throwable e) {
            System.out.println("Ошибка просмотра таблицы: " +
e.getMessage());
            System.out.println("Вероятно, не найден JAR Berkeley JE.
Поместите je-*.jar в папку 'Lab3/lib'.");
        }
    }

    private static void pause(Scanner sc) {
        System.out.print("Нажмите Enter, чтобы вернуться в меню...");
        try { sc.nextLine(); } catch (Exception ignored) {}
    }

```

```

        System.out.println();
    }

    private static void saveTable(String env, String table, Path
outFile) {
        PrintStream orig = System.out;
        try (PrintStream ps = new
PrintStream(Files.newOutputStream(outFile), true,
StandardCharsets.UTF_8)) {
            System.setOut(ps);
            try {
                if (!Files.exists(Paths.get(env))) {
                    orig.println("Среда Berkeley DB не найдена: " +
Paths.get(env).toAbsolutePath());
                    orig.println("Запустите конвертацию (пункт 1) и
повторите.");
                } else {
                    DumpBerkeley.main(new String[]{"--env", env,
table});
                }
            } catch (Throwable e) {
                orig.println("Ошибка сохранения таблицы: " +
e.getMessage());
                orig.println("Вероятно, не найден JAR Berkeley JE.
Поместите je-*.jar в папку 'Lab3/lib'.");
            }
        } catch (IOException e) {
            orig.println("Ошибка записи файла: " + e.getMessage());
        } finally {
            System.setOut(orig);
        }
    }

    private static void printMenu() {
        System.out.println();
        System.out.println("Выберите действие:");
        System.out.println("  1. Запустить конвертацию (PostgreSQL →
Berkeley DB)");
        System.out.println("  2. Показать список таблиц");
        System.out.println("  3. Показать таблицу");
        System.out.println("  4. Сохранить таблицу в файл (UTF-8)");
        System.out.println("  0. Выход");
        System.out.print("Ваш выбор: ");
    }

    public static void main(String[] args) {
        try {
            System.setOut(new PrintStream(new BufferedOutputStream(new
FileOutputStream(FileDescriptor.out)), true, StandardCharsets.UTF_8));
            System.setErr(new PrintStream(new BufferedOutputStream(new
FileOutputStream(FileDescriptor.err)), true, StandardCharsets.UTF_8));
        } catch (Exception ignore) {}
        // Базовая папка — та, где лежит AppMenu.class (Lab3)
        Path baseDir = getBaseDir();
        String env = resolveEnv(baseDir);
    }

```

```

Scanner sc = new Scanner(System.in, StandardCharsets.UTF_8);

while (true) {
    printMenu();
    String choice = sc.nextLine().trim();
    switch (choice) {
        case "1":
            System.out.println("→ Запуск конвертации...");
            runConverter(baseDir);
            System.out.println("✓ Конвертация завершена");
            pause(sc);
            break;
        case "2":
            System.out.println("→ Список таблиц (env=" + env +
                "):");
            listTables(env);
            pause(sc);
            break;
        case "3":
            System.out.print("Введите имя таблицы: ");
            String table = sc.nextLine().trim();
            if (table.isEmpty()) {
                System.out.println("Имя таблицы не задано");
                break;
            }
            System.out.println("→ Печать таблицы '" + table +
                "' (env=" + env + "):");
            showTable(env, table);
            pause(sc);
            break;
        case "4":
            System.out.print("Имя таблицы: ");
            String t = sc.nextLine().trim();
            if (t.isEmpty()) { System.out.println("Имя таблицы
не задано"); break; }
            System.out.print("Путь файла (например, view_" + t
                + ".txt): ");
            String path = sc.nextLine().trim();
            if (path.isEmpty()) { System.out.println("Путь
файла не задан"); break; }
            Path out = baseDir.resolve(path);
            System.out.println("→ Сохранение в файл: " +
                out.toAbsolutePath());
            saveTable(env, t, out);
            System.out.println("✓ Готово");
            pause(sc);
            break;
        case "0":
            System.out.println("Выход");
            return;
        default:
            System.out.println("Неизвестный выбор: '" + choice
                + "'. Попробуйте снова.");
    }
}

```



```
}
```

Файл config.json

```
{
  "java_tool_options": "-Dfile.encoding=UTF-8",
  "logs_mode": "utf8",
  "berkeley_env": "./berkeley_data",
  "jdk_bin": "./lib/jdk-17.0.17+10/bin",
  "je_jar": "./lib/je-18.3.12.jar",
  "postgres_jar": "./lib/postgresql-42.7.3.jar"
}
```

2.5 Код файла DumpBerkeley.java

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;

import com.sleepycat.je.*;

/**
 * DumpBerkeley — простой просмотрщик для Berkeley DB JE.
 * Печатает пары key/value как UTF-8 (без hex/\xNN),
 * чтобы удобно смотреть JSON-значения.
 *
 * Использование:
 *   java -cp <Lab3>;<je.jar> DumpBerkeley [--env <path>] [--out
 <file>] <dbName>
 *   java -cp <Lab3>;<je.jar> DumpBerkeley [--env <path>] --list
 */
public class DumpBerkeley {
    public static void main(String[] args) throws Exception {
        Path envPath = Paths.get("Lab3", "berkeley_data");
        String dbName = null;
        Path outFile = null;
        boolean listOnly = false;

        for (int i = 0; i < args.length; i++) {
            String a = args[i];
            switch (a) {
                case "--env":
                    envPath = Paths.get(args[++i]);
                    break;
                case "--out":
                    outFile = Paths.get(args[++i]);
                    break;
                case "--list":
                    listOnly = true;
                    break;
                default:
                    dbName = a;
            }
        }
    }
}
```

```

        if (!Files.exists(envPath)) {
            throw new IllegalStateException("Environment not found: "
+ envPath.toAbsolutePath());
        }

        EnvironmentConfig cfg = new EnvironmentConfig();
        cfg.setAllowCreate(false);
        try (Environment env = new Environment(envPath.toFile(), cfg))
        {
            if (listOnly) {
                for (String name : env.getDatabaseNames()) {
                    System.out.println(name);
                }
                return;
            }

            if (dbName == null) {
                throw new IllegalArgumentException("Usage:
DumpBerkeley [--env <path>] [--out <file>] <dbName> | --list");
            }

            DatabaseConfig dc = new DatabaseConfig();
            dc.setAllowCreate(false);
            try (Database db = env.openDatabase(null, dbName, dc)) {
                boolean toFile = (outFile != null);
                if (toFile) {
                    try (BufferedWriter bw =
Files.newBufferedWriter(outFile, StandardCharsets.UTF_8)) {
                        bw.write("VERSION=viewer\n");
                        bw.write("db=" + dbName + "\n");
                        bw.write("HEADER=END\n");
                        Cursor cursor = db.openCursor(null, null);
                        DatabaseEntry k = new DatabaseEntry();
                        DatabaseEntry v = new DatabaseEntry();
                        while (cursor.getNext(k, v, LockMode.DEFAULT)
== OperationStatus.SUCCESS) {
                            String key = new String(k.getData(),
StandardCharsets.UTF_8);
                            String val = new String(v.getData(),
StandardCharsets.UTF_8);
                            bw.write(key);
                            bw.write("\n");
                            bw.write(val);
                            bw.write("\n");
                        }
                        cursor.close();
                        bw.flush();
                    }
                } else {
                    BufferedWriter bw = new BufferedWriter(new
OutputStreamWriter(System.out, StandardCharsets.UTF_8));
                    bw.write("VERSION=viewer\n");
                    bw.write("db=" + dbName + "\n");
                    bw.write("HEADER=END\n");
                    Cursor cursor = db.openCursor(null, null);
                    DatabaseEntry k = new DatabaseEntry();

```

```

        DatabaseEntry v = new DatabaseEntry();
        while (cursor.getNext(k, v, LockMode.DEFAULT) ==
OperationStatus.SUCCESS) {
            String key = new String(k.getData(),
StandardCharsets.UTF_8);
            String val = new String(v.getData(),
StandardCharsets.UTF_8);
            bw.write(key);
            bw.write("\n");
            bw.write(val);
            bw.write("\n");
        }
        cursor.close();
        bw.flush(); // Не закрываем System.out
    }
}
}
}
}

```

Файл PgToBerkeley.java

```

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.sql.*;
import java.util.*;

// Berkeley DB Java Edition
import com.sleepycat.je.Database;
import com.sleepycat.je.DatabaseConfig;
import com.sleepycat.je.DatabaseEntry;
import com.sleepycat.je.Environment;
import com.sleepycat.je.EnvironmentConfig;

public class PgToBerkeley {
    // Определяем рабочую папку динамически от текущего каталога
    запуска
    private static final Path WORKSPACE =
Paths.get("").toAbsolutePath();
    private static final Path CONFIG_PY =
WORKSPACE.resolve("config.py");
    private static final Path LAB3_DIR = WORKSPACE.resolve("Lab3");
    private static final Path OUTPUT_DIR =
LAB3_DIR.resolve("berkeley_data");
    private static final boolean UNICODE_LOGS =
Optional.ofNullable(System.getenv("PG2B_LOGS"))
        .map(v -> v.equalsIgnoreCase("unicode")) ||
v.equalsIgnoreCase("utf8"))
        .orElse(false);
    private static PrintWriter LOG;

    public static void main(String[] args) {
        // Принудительно выставляем UTF-8 для консольного вывода,
        // чтобы кириллица не превращалась в "?????" на Windows-
        консоли.
    }
}

```

```

        try {
            System.setOut(new PrintStream(new BufferedOutputStream(new
FileOutputStream(FileDescriptor.out)), true, StandardCharsets.UTF_8));
            System.setErr(new PrintStream(new BufferedOutputStream(new
FileOutputStream(FileDescriptor.err)), true, StandardCharsets.UTF_8));
        } catch (Exception ignore) {}
        try {
            Files.createDirectories(LAB3_DIR);
            LOG = new PrintWriter(Files.newBufferedWriter(
                LAB3_DIR.resolve("converter.log"),
                StandardCharsets.UTF_8,
                StandardOpenOption.CREATE,
                StandardOpenOption.TRUNCATE_EXISTING
            ));
        } catch (IOException ioe) {
            LOG = null; // fallback to console-only
        }
        log("[PgToBerkeley] Conversion start...");
        try {
            Map<String, String> cfg = parsePythonConfig(CONFIG_PY);
            String host = cfg.getDefault("host", "127.0.0.1");
            String port = cfg.getDefault("port", "5432");
            String db = cfg.getDefault("database", "postgres");
            String user = cfg.getDefault("user", "postgres");
            String pass = cfg.getDefault("password", "");

            String url = "jdbc:postgresql://" + host + ":" + port +
"/" + db;
            log("[PgToBerkeley] Connecting to " + url);

            Properties props = new Properties();
            props.setProperty("user", user);
            props.setProperty("password", pass);
            try (Connection conn = DriverManager.getConnection(url,
props)) {
                Files.createDirectories(OUTPUT_DIR);
                convertAllTables(conn, OUTPUT_DIR);
            }

            log("[PgToBerkeley] Conversion finished successfully.");
        } catch (Exception e) {
            logErr("[PgToBerkeley] Error: " +
asciiSafe(e.getMessage()));
            e.printStackTrace();
            System.exit(1);
        } finally {
            if (LOG != null) {
                try { LOG.flush(); LOG.close(); } catch (Exception
ignore) {}
            }
        }
    }

    private static Map<String, String> parsePythonConfig(Path path)
throws IOException {
        Map<String, String> out = new HashMap<>();

```

```

        if (!Files.exists(path)) {
            throw new FileNotFoundException("config.py not found: " +
path.toString());
        }
        List<String> lines = Files.readAllLines(path,
StandardCharsets.UTF_8);
        // Простой парсер DB_CONFIG = { 'host': '...', "port":
5432, ... }
        for (String line : lines) {
            String l = line.trim();
            if (l.startsWith("'host'")) {
                out.put("host", extractValue(l));
            } else if (l.startsWith("\"host\"")) {
                out.put("host", extractValue(l));
            } else if (l.startsWith("'port'") ||
l.startsWith("\"port\"")) {
                out.put("port", extractValue(l));
            } else if (l.startsWith("'database'") ||
l.startsWith("\"database\"")) {
                out.put("database", extractValue(l));
            } else if (l.startsWith("'user'") ||
l.startsWith("\"user\"")) {
                out.put("user", extractValue(l));
            } else if (l.startsWith("'password'") ||
l.startsWith("\"password\"")) {
                out.put("password", extractValue(l));
            }
        }
        return out;
    }

    private static String extractValue(String line) {
        // ожидаем формат: 'key': 'value', или "key": "value",
        String[] parts = line.split(":", 2);
        if (parts.length < 2) return "";
        String v = parts[1].trim();
        // убрать завершающую запятую
        if (v.endsWith(",")) v = v.substring(0, v.length() -
1).trim();
        // убрать кавычки
        if ((v.startsWith("'") && v.endsWith("'")) ||
(v.startsWith("\"") && v.endsWith("\""))) {
            v = v.substring(1, v.length() - 1);
        }
        return v;
    }

    private static void convertAllTables(Connection conn, Path envDir)
throws Exception {
        List<String> tables = listPublicTables(conn);
        if (tables.isEmpty()) {
            log("[PgToBerkeley] No tables in public schema.");
            return;
        }

        EnvironmentConfig envCfg = new EnvironmentConfig();

```

```

        envCfg.setAllowCreate(true);
        try (Environment env = new Environment(envDir.toFile(),
envCfg)) {
            for (String table : tables) {
                List<String> pkCols = listPrimaryKeyColumns(conn,
table);
                if (pkCols.isEmpty()) {
                    log("[WARN] Table '" + displayName(table) + "'
skipped: no primary key.");
                    continue;
                }

                List<String> allCols = listColumns(conn, table);
                List<String> valueCols = new ArrayList<>();
                for (String c : allCols) {
                    if (!pkCols.contains(c)) valueCols.add(c);
                }
                if (valueCols.isEmpty()) {
                    log("[WARN] Table '" + displayName(table) + "'
skipped: no value columns.");
                    continue;
                }

                DatabaseConfig dbCfg = new DatabaseConfig();
                dbCfg.setAllowCreate(true);
                String dbName = table; // one JE database per table
                try (Database db = env.openDatabase(null, dbName,
dbCfg)) {
                    long written = 0;
                    String quotedTable = "'" + table + "'";
                    String sql = "SELECT * FROM public." +
quotedTable;

                    try (Statement st =
conn.createStatement(ResultSet.TYPE_FORWARD_ONLY,
ResultSet.CONCUR_READ_ONLY)) {
                        st.setFetchSize(1000);
                        try (ResultSet rs = st.executeQuery(sql)) {
                            ResultSetMetaData md = rs.getMetaData();
                            int colCount = md.getColumnCount();
                            while (rs.next()) {
                                String key = buildKey(rs, pkCols, md);
                                String json = buildJson(rs, valueCols,
md);

                                DatabaseEntry k = new
DatabaseEntry(key.getBytes(StandardCharsets.UTF_8));
                                DatabaseEntry v = new
DatabaseEntry(json.getBytes(StandardCharsets.UTF_8));
                                db.put(null, k, v);
                                written++;
                            }
                        }
                    }
                    log(String.format("[PgToBerkeley] Table %s ->
rows: %d", displayName(table), written));
                }
            }
        }

```

```

    }
}

private static List<String> listPublicTables(Connection conn)
throws SQLException {
    String q = "SELECT table_name FROM information_schema.tables
WHERE table_schema='public' AND table_type='BASE TABLE' ORDER BY
table_name";
    List<String> out = new ArrayList<>();
    try (Statement st = conn.createStatement(); ResultSet rs =
st.executeQuery(q)) {
        while (rs.next()) out.add(rs.getString(1));
    }
    return out;
}

private static List<String> listPrimaryKeyColumns(Connection conn,
String table) throws SQLException {
    String q = "SELECT kcu.column_name\n" +
        "FROM information_schema.table_constraints tc\n" +
        "JOIN information_schema.key_column_usage kcu\n" +
        "ON tc.constraint_name = kcu.constraint_name AND
tc.table_schema = kcu.table_schema\n" +
        "WHERE tc.table_schema='public' AND tc.table_name=?
AND tc.constraint_type='PRIMARY KEY'\n" +
        "ORDER BY kcu.ordinal_position";
    List<String> out = new ArrayList<>();
    try (PreparedStatement ps = conn.prepareStatement(q)) {
        ps.setString(1, table);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) out.add(rs.getString(1));
        }
    }
    return out;
}

private static List<String> listColumns(Connection conn, String
table) throws SQLException {
    String q = "SELECT column_name FROM information_schema.columns
WHERE table_schema='public' AND table_name=? ORDER BY
ordinal_position";
    List<String> out = new ArrayList<>();
    try (PreparedStatement ps = conn.prepareStatement(q)) {
        ps.setString(1, table);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) out.add(rs.getString(1));
        }
    }
    return out;
}

private static String buildKey(ResultSet rs, List<String> pkCols,
ResultSetMetaData md) throws SQLException {
    // Если ПК составной, ключ = значения через '|'
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < pkCols.size(); i++) {

```

```

        String col = pkCols.get(i);
        Object val = rs.getObject(col);
        if (val != null) sb.append(val.toString());
        if (i < pkCols.size() - 1) sb.append('|');
    }
    return sb.toString();
}

private static String buildJson(ResultSet rs, List<String> cols,
ResultSetMetaData md) throws SQLException {
    StringBuilder sb = new StringBuilder();
    sb.append('{');
    for (int i = 0; i < cols.size(); i++) {
        String col = cols.get(i);
        Object val = rs.getObject(col);

sb.append('\"').append(escapeJson(col)).append('\"').append(':');
        if (val == null) {
            sb.append("null");
        } else if (val instanceof Number) {
            sb.append(val.toString());
        } else if (val instanceof Boolean) {
            sb.append(((Boolean) val) ? "true" : "false");
        } else {

sb.append('\"').append(escapeJson(val.toString())).append('\"');
        }
        if (i < cols.size() - 1) sb.append(',');
    }
    sb.append('}');
    return sb.toString();
}

private static String escapeJson(String s) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        switch (c) {
            case '\\': sb.append("\\\\"); break;
            case '\"': sb.append("\\\""); break;
            case '\n': sb.append("\\n"); break;
            case '\r': sb.append("\\r"); break;
            case '\t': sb.append("\\t"); break;
            default:
                if (c < 0x20) {
                    sb.append(String.format("\\u%04x", (int)c));
                } else {
                    sb.append(c);
                }
            }
    }
    return sb.toString();
}

private static String asciiSafe(String s) {
    if (s == null) return "";

```



```

        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (c <= 0x7F) {
                sb.append(c);
            } else {
                sb.append(String.format("\\u%04X", (int) c));
            }
        }
        return sb.toString();
    }

    private static String displayName(String s) {
        return UNICODE_LOGS ? s : asciiSafe(s);
    }

    private static void log(String msg) {
        System.out.println(msg);
        if (LOG != null) {
            LOG.println(msg);
        }
    }

    private static void logErr(String msg) {
        System.err.println(msg);
        if (LOG != null) {
            LOG.println(msg);
        }
    }
}

```

Файл run_viewer.ps1

```

param(
    [switch]$List,
    [string]$Table,
    [switch]$All,
    [string]$OutDir
)

$ErrorActionPreference = 'Stop'
$here = $PSScriptRoot

try { [Console]::OutputEncoding = [System.Text.Encoding]::UTF8 } catch {}

chcp 65001 > $null

# Загружаем конфиг
$configPath = Join-Path $here 'config.json'
if (!(Test-Path $configPath)) {
    throw "Не найден файл конфигурации: $configPath"
}
$cfg = Get-Content $configPath -Raw | ConvertFrom-Json

function Resolve([string]$p) {
    [System.IO.Path]::GetFullPath((Join-Path $here $p))
}

```

```

}

$jdkBin = Resolve $cfg.jdk_bin
$jeJar  = Resolve $cfg.je_jar
$envDir = Resolve $cfg.berkeley_env

$env:PATH = "$jdkBin;$env:PATH"
if ($cfg.java_tool_options) { $env:JAVA_TOOL_OPTIONS =
$cfg.java_tool_options }

Write-Host "Компиляция DumpBerkeley.java..." -ForegroundColor Cyan
& "$jdkBin/javac.exe" -encoding UTF-8 -cp "$jeJar" (Join-Path $here
'DumpBerkeley.java')

function RunJava([string[]]$args) {
    & "$jdkBin/java.exe" @args
}

if ($List) {
    RunJava -args @('-cp', "$here;$jeJar", 'DumpBerkeley', '--env',
$envDir, '--list')
    exit 0
}

if ($All) {
    $tables = RunJava -args @('-cp', "$here;$jeJar", 'DumpBerkeley', '--
env', $envDir, '--list')
    if (-not $OutDir) { $OutDir = $here }
    foreach ($t in $tables) {
        RunJava -args @('-cp', "$here;$jeJar", 'DumpBerkeley', '--env',
$envDir, $t) |
        Out-File -FilePath (Join-Path $OutDir ("view_{0}.txt" -f $t)) -
Encoding utf8
    }
    Write-Host ("Сохранено: {0} таблиц в '{1}'" -f $tables.Count,
(Resolve-Path $OutDir)) -ForegroundColor Green
    exit 0
}

if ($Table) {
    if ($OutDir) {
        RunJava -args @('-cp', "$here;$jeJar", 'DumpBerkeley', '--env',
$envDir, $Table) |
        Out-File -FilePath (Join-Path $OutDir ("view_{0}.txt" -f
$Table)) -Encoding utf8
        Write-Host ("Сохранено в файл: {0}" -f (Join-Path $OutDir
("view_{0}.txt" -f $Table))) -ForegroundColor Green
    } else {
        RunJava -args @('-cp', "$here;$jeJar", 'DumpBerkeley', '--env',
$envDir, $Table)
    }
    exit 0
}

# По умолчанию список

```

```
RunJava -args @('-cp', "$here;$jeJar", 'DumpBerkeley', '--env',  
$envDir, '--list')
```

Файл config.py

```
# Database connection settings  
DB_CONFIG = {  
    'host': 'localhost',  
    'port': 5432,  
    'database': 'Railway',  
    'user': 'postgres',  
    'password': '1111'  
}  
  
# Superuser password  
SUPERUSER_PASSWORD = 'super'  
  
# List of lookup tables that can only be edited by a superuser  
LOOKUP_TABLES = [  
    "Маршрут",  
    "Станция",  
    "Поезд",  
    "Сотрудник"  
]
```